

Numerical Optimization for VITESS simulations- Version 2

Principle

The combination of Monte Carlo simulations and optimization is realized in a way that the optimization routine is the main program and calls the simulation in a subroutine called *ExtFunctions()* (see Fig. 1).

This procedure first writes a file *Pcomm.dat* containing one or more parameter sets. The tool *gener_pipe.exe* reads these sets and combines them with the standard instrument (saved as *std_instr.cmd*), thus generating the VITESS pipe commands (in a shell or batch script) for all simulations to perform.

The simulations are then started and the files (of all simulations) that are needed for the figure of merit are copied. The next step is to start a program to calculate the figures-of-merit (for all simulations) and write them into a file *Fcomm.dat*. Now this/these figure(s)-of-merit are read and returned to the optimization program. This will suggest (a) new parameter set(s) and call *ExtFunctions* again. This procedure continues until the optimization has come to an end.

The optimization is widely controlled by the 3 ASCII input files '*opt_param.ini*', '*sim_param.ini*' and '*fom.ini*' and an ASCII control file for the optimization used. If there is a 1 to 1 relation between optimization and simulation parameters and the figure of merit is one of the many options of the delivered *fom* program, no code or script is necessary for the optimization.

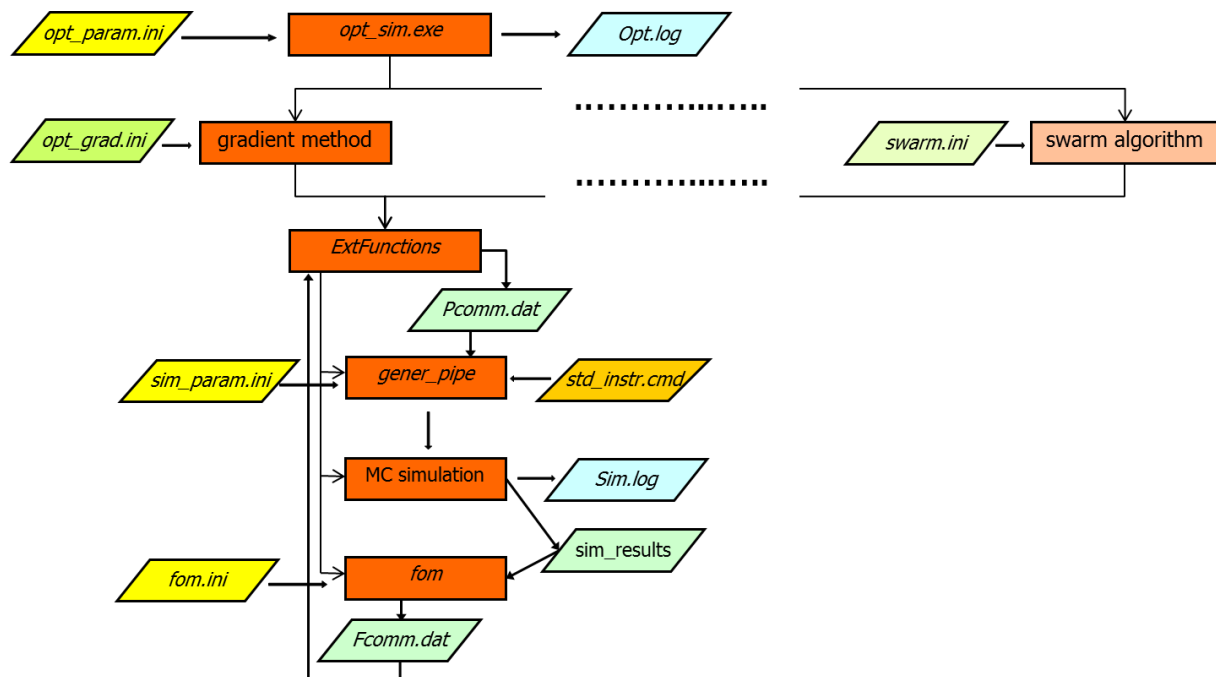


Figure 1: Scheme of the optimization

Directories

The directory ‘... VitessX.Y/MODULES’ where the executables are will be called the **module directory**.

The directory ‘... VitessX.Y/OPTIMIZATON’ contains all necessary input files for the optimization. You can run the optimization here or choose a different folder. This will be called **optimization directory**.

The simulations generally run in a different directory. This can be the directory where the instrument has been developed or another one. This will be called **simulation directory**.

7 Steps to the Optimization

Step 1: Necessary Files

The executable **opt_sim** has be copied from the module directory to the optimization directory. If an optimization directory different from ... VitessX.Y/OPTIMIZATON is chosen, all ini files have to be copied from there to optimization directory. If you choose a simulation directory that is not the original directory, you have to copy **all input files** there.

Step 2: Standard Instrument

The first step is to develop an instrument that can be used as a starting point for the optimization. Make sure that the simulation runs properly and that an adequate number of trajectories is chosen; at least 100'000 trajectories should contribute to the figure of merit, i.e. have to be summed up in the file used to calculate the figure of merit.

When it is finished it needs to be saved by clicking “Check” and saving the shown output without any change by using the “Save” button (belonging to the output window) as **std_instr.cmd**. to the **optimization directory!!!** Note that this command file cannot be used for an optimization on another platform, (e.g. saved under Windows and used under Unix) and that it has to be **left in this format!!!** (Of course editing this file, e.g. changing the number of trajectories, is possible.)

Now the different ini-files have to be filled. Note that anything behind a #-symbol is regarded as a comment and ignored by the program.

Step 3: Optimization Parameters

In the file ‘opt_param.ini’ you have to give

1. the name of the optimization
2. a code word to define the optimization algorithm
existing at the moment : 'opt_grad', 'opt_grad_mc', 'metropolis' (see chapter ‘Algorithms’ at the end)
3. option for parallel computing
4. all input variables in the form
 - index (starting with 1)
 - initial value
 - minimal value

- maximal value
- difference to the actual value for the calculation of the gradient or the next tested value

Notes:

- Not all algorithms will need all information about the parameters or can treat them, but for the sake of simplicity this needs to be given for all of them.
- Input lines must not be removed and their order has to be kept.
- For gradient methods, the difference 'delta' should be only a fraction of the expected value to calculate the differential quotient (for the actual value). For other methods this limits the step width to the next position and should therefore be in the order of magnitude of the value or a bit smaller.
- Parallel computing is foreseen for optimizations and this option has to be set already, but it is not yet realized. .

```

opt_param.ini - Editor
Datei Bearbeiten Format Ansicht ?
Optimisation of the sample position # name of the optimization
#
metropolis # algorithm existing: 'opt_grad_mc', 'metropolis', 'opt_grad'
parallel # parallel option existing: 'parallel' (used for MC simulations), 'sequential' (to be used for fitting)
#
# parameters
# start min max delta description
1 2.0 -10.0 10.0 1.0 # P1: hor. shift [cm]
2 2.0 -10.0 10.0 1.0 # P2: vert. shift [cm]

```

```

opt_param.ini - Editor
Datei Bearbeiten Format Ansicht ?
Optimisation of the sample position # name of the optimization
#
opt_grad_mc # algorithm existing: 'opt_grad_mc', 'metropolis', 'opt_grad'
parallel # parallel option existing: 'parallel' (used for MC simulations), 'sequential' (to be used for fitting)
#
# parameters
# start min max delta description
1 2.0 -10.0 10.0 0.05 # P1: hor. shift [cm]
2 2.0 -10.0 10.0 0.05 # P2: vert. shift [cm]

```

File 1: file *opt_param.ini* setting optimization algorithm and parameters to be optimized

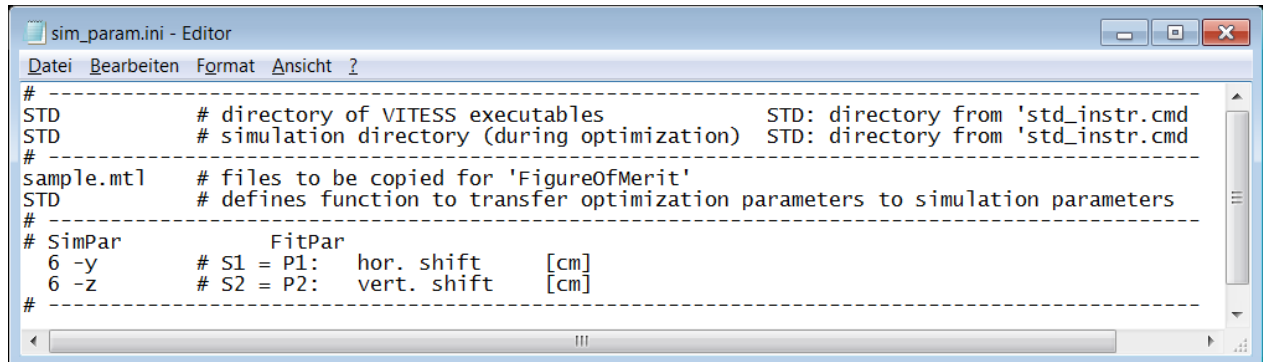
Step 4: Control of the Simulation

The first 2 lines in *sim_param.ini* define the directory where the executables can be found (and thus the Vitess version) – line 1 – and the simulation directory (cf. section ‘Directories’) – line 2 (see Files 2a and 2b). STD means that the path to this directory is taken from the standard instrument file ‘std_instr.cmd’

The third line defines the file(s) necessary to calculate the figure of merit (see File 2a, cf. Step 4).

The next line of this file defines the function calculating the simulation parameters S_k from the optimization parameters P_j . STD defines a direct assignment: $S_1 = P_1$, $S_2 = P_2$, etc. In the future, simple mathematical operations between these parameters can be defined in the way indicated in the file. For the time being, only this direct assignment is possible, everything else has to be realized by extending the program *gener_pipe* - see below.

In the next lines, all simulation parameters affected by the optimization parameters are listed (see File 2a,b). They are characterized by the module number and the symbol (like “-H”) defining the parameter (within the list of existing parameters of this module). The symbol is shown if you click on the text belonging to this item.



```

sim_param.ini - Editor
Datei Bearbeiten Format Ansicht ?
# -----
STD          # directory of VITESS executables          STD: directory from 'std_instr.cmd'
STD          # simulation directory (during optimization) STD: directory from 'std_instr.cmd'
# -----
sample.mtl   # files to be copied for 'FigureOfMerit'
STD          # defines function to transfer optimization parameters to simulation parameters
# -----
# SimPar      FitPar
6 -y          # S1 = P1: hor. shift      [cm]
6 -z          # S2 = P2: vert. shift    [cm]
# -----

```

File 2a: file *sim_param.ini* defining the variable simulation parameters and the function transferring the optimization parameters to simulation parameters.

In order to calculate simulation parameters from optimization parameters, a function similar to *DetSimParamStd()* has to be written in *gener_pipe.c*, which performs this calculation. (The easiest and safest way is the following: copy, rename and change *DetSimParamStd()*. Then add 3 lines of code in the *if...else if...else* command identifying this function by code that will then be given in *sim_param.ini*. instead of STD.

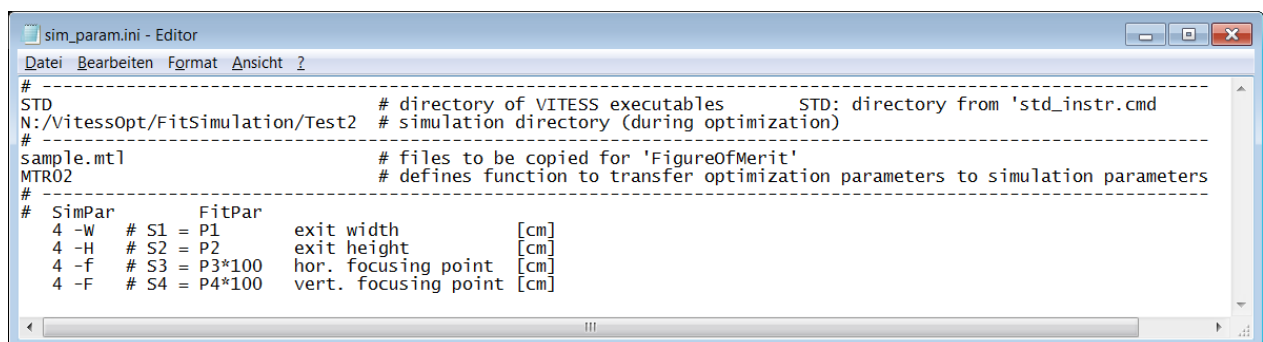
Example: Adding

```

else if (strcmp(sParFct, "MTR02")==0)
{
    DetSimParamMtr(P, m, nSimParNo, nFitParNo, 2);
}

```

assigns that the function *DetSimParamMtr()* with additional parameter *iFirstMPar=2* is called. It transfers the optimization parameters to simulation parameters, if the code ‘MTR02’ is given in the fourth line of *sim_param.ini*. (This has in fact been realized for the optimization of the elliptical guide exit of the EXED instrument, cf. File 2b. MTR denotes the change from meter to cm and 02 indicates for which simulation parameters this is applied.)



```

sim_param.ini - Editor
Datei Bearbeiten Format Ansicht ?
# -----
STD          # directory of VITESS executables          STD: directory from 'std_instr.cmd'
N:/VitessOpt/FitSimulation/Test2 # simulation directory (during optimization)
# -----
sample.mtl   # files to be copied for 'FigureOfMerit'
MTR02        # defines function to transfer optimization parameters to simulation parameters
# -----
# SimPar      FitPar
4 -W          # S1 = P1      exit width      [cm]
4 -H          # S2 = P2      exit height     [cm]
4 -f          # S3 = P3*100  hor. focusing point [cm]
4 -F          # S4 = P4*100  vert. focusing point [cm]
# -----

```

File 2b: special version of *sim_param.ini* where the function *DetSimParamMtr* is used for the transfer.

Step 5: The Figure of Merit

The next step is to assess the figure of merit. The function ‘*ExtFunctions()*’ in *calc_sim_fom.c* will call the program *fom*; this writes the file *Fcomm.dat*. It is an ASCII file containing just one number for each simulation to be executed. In case of N simulations, these N numbers are

written in N rows. The files defined in *sim_param.ini* – in this example *sample.mtl* - serve as a data basis for these calculations.

In the future, using user defined programs will be made easier. In the present version, it has to be called *fom* (plus extension) and be written to the folder MODULES replacing the existing program, or the name in *ExtFunctions()* has to be changed and *opt_sim* re-compiled. *fom* is delivered with the package. It is controlled via *fom.ini*. The general formulae are

$$FoM = \frac{\sum_i \frac{w_i I_{sign,i} \lambda_i^l}{I_{ref,i}^m}}{\sum_i I_{noise,i}^n}$$

or

$$FoM = \frac{\langle \frac{w_i I_{sign,i} \lambda_i^l}{I_{ref,i}^m} \rangle}{\langle I_{noise,i}^n \rangle}$$

Where the parameter A defines if sums (A=0) or averages (A=1) are used. The numbers for the powers l,m,n have to be given; they can be float values. The files containing signal, noise, reference and weight have to be given. (If they output files of the simulations they have to be listed in *sim.ini* as well. If a weighting with wavelength shall be used, the intensity files have to be intensities as a function of wavelength.

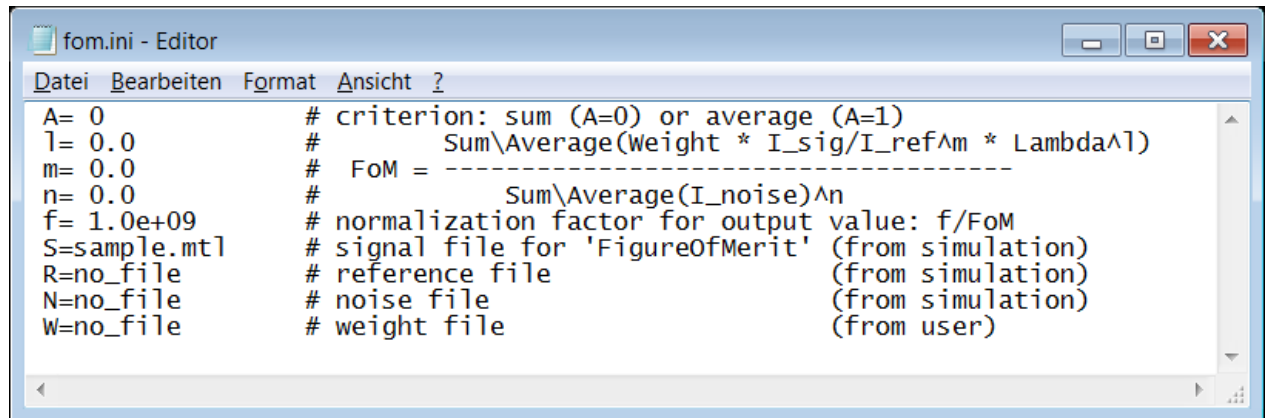
As the optimization routine searches for the minimum, the value f/FoM is written to *Pcomm.dat*. The factor is also read from *fom.ini* and allows setting output values to the order of magnitude 1, which is an advantage (in terms of numerical precision) for some optimization algorithms.

In the example shown in File 7, the figure of merit is reduced to

$$FoM = \sum I_{sign,i}$$

thus optimizing the integrated flux (in the slit).

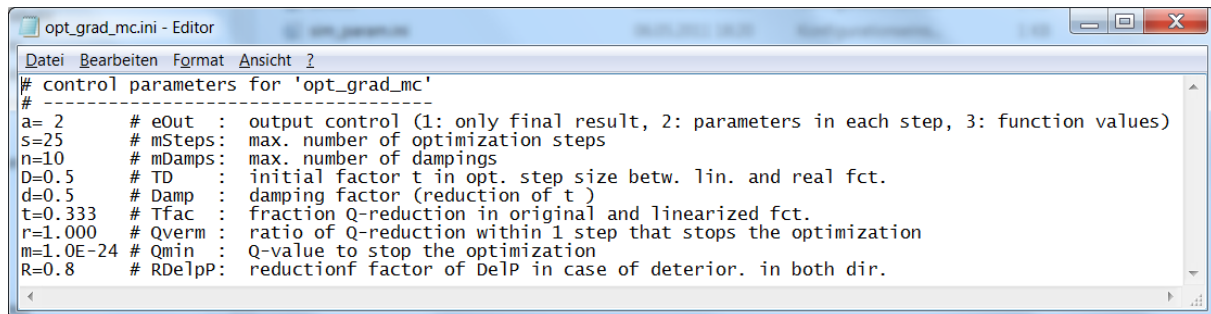
Another example: if the transmission averaged over a wavelength range shall be optimized, the parameters have to be set as: A=1; m=1; l=n=0 and the reference file (=input of the section, whose transmission is regarded) has to be given in addition to the signal file.



File 3: file *fom.ini* to define the figure of merit using the program *fom_lambda.exe*.

Step 6: Control of the optimization routine

The optimization process is controlled by the file 'opt_name.ini', i.e. each algorithm has a control file of its own. Useful default parameters are delivered so that usually no changes are necessary. Changing these parameters is possible, but requires knowledge of the algorithm.

A screenshot of a text editor window titled 'opt_grad_mc.ini - Editor'. The window contains a configuration file for the 'opt_grad_mc' algorithm. The file starts with a comment line '# control parameters for \'opt_grad_mc\'' followed by a dashed line separator. Below this, various parameters are listed with their values and descriptions. The parameters include 'a' (output control), 's' (max. number of optimization steps), 'n' (max. number of dampings), 'D' (initial factor t), 'd' (damping factor), 't' (fraction Q-reduction), 'r' (ratio of Q-reduction), 'm' (Q-value to stop), and 'R' (reduction factor of DelP).

```
# control parameters for 'opt_grad_mc'
# -----
a= 2      # eOut : output control (1: only final result, 2: parameters in each step, 3: function values)
s=25      # mSteps: max. number of optimization steps
n=10      # mDamps: max. number of dampings
D=0.5     # TD : initial factor t in opt. step size betw. lin. and real fct.
d=0.5     # Damp : damping factor (reduction of t)
t=0.333   # Tfac : fraction Q-reduction in original and linearized fct.
r=1.000   # Qverm : ratio of Q-reduction within 1 step that stops the optimization
m=1.0E-24 # Qmin : Q-value to stop the optimization
R=0.8     # RDelP: reductionf factor of DelP in case of deterior. in both dir.
```

File 4: file *opt_grad_mc.ini* controlling a gradient method adapted to Monte Carlo simulations

Step 7: Run

Now the optimization can be run by starting the program *opt_sim* in the optimization directory.

Output

A log file **Opt.log** of the optimization is written to the optimization directory. It contains the initial and final parameter values and (depending on the algorithm) all or some of the parameters sets in between as well as the figures of merit for these parameter sets. Additionally, log files **Sim0.log**, **Sim1.log** ... for the different simulations of the last optimization step can be found in the simulation directory. Finally, the files contributing to the figure of merit are copied. So those files obtained in the final optimization step still exist.

Algorithms

In version 3.1 three algorithms are available, others will follow in upcoming versions: a usual and an adapted gradient method as well as a metropolis algorithm.

The usual gradient method chosen by 'opt_grad' was developed for fitting and works there very well in practically all cases.

It has been changed to allow for the statistical effects in combination with Monte Carlo simulations. This adapted gradient method is used when choosing 'opt_grad_mc'.

Both gradient methods work best, if the parameter are all in a similar order of magnitude and especially not larger than 10^6 . The gradient is determined numerically varying one parameter in both direction and calculating the differential quotient. Care should be taken in defining the parameter variation: it should not be too large to get a good estimation of the gradient for this parameter value, but especially not too small to avoid zero difference and a statistically induced false estimation.

'opt_grad_mc' allows setting minimal and maximal value for a parameter, but the optimization runs better if this option is not used.

Generally, gradient methods work well with Monte Carlo simulations, if a sufficient number of events – ideally up to 10^6 - contribute to the figure of merit.

The Metropolis algorithm is used if 'metropolis' is chosen. The risk of finding a local instead of the global optimum is reduced by using the metropolis algorithm; but it needs by far more simulations, at least 1000 steps. On the other hand, it allows getting an estimate of the width of the local optimum by calculating centre and variance over the path in the local optimum.